

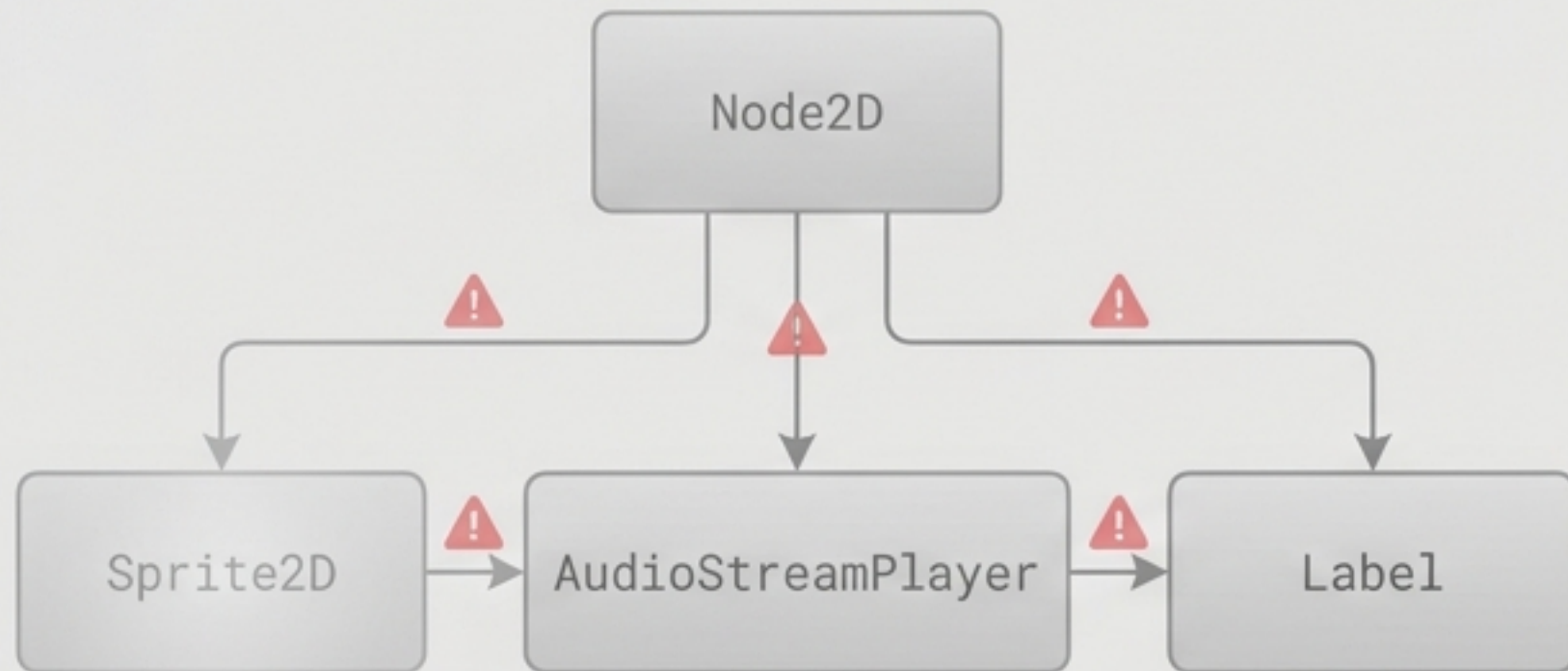


Arquitectura Limpia en Godot: Diseñando un Motor de Invasores del Espacio en C#

Una deconstrucción modular de la lógica, el renderizado y la separación de responsabilidades.

// AUTOR: C. SISTEMAS | OCTUBRE 2023 | Lenguaje: C# 10.0 | Motor: Godot 4.x

El Enfoque Tradicional



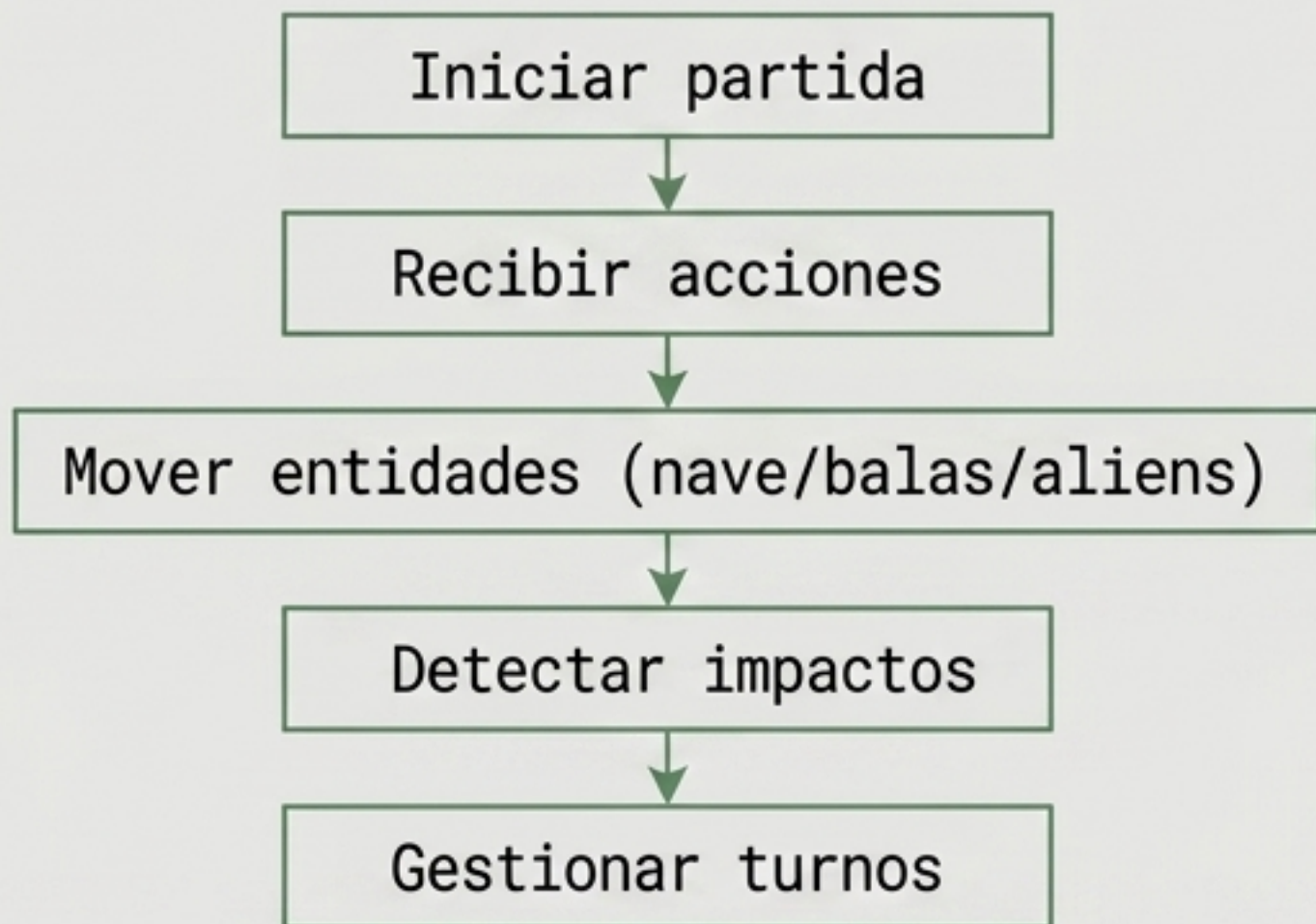
La Filosofía del Proyecto

Cero Recursos Externos. Código Puro.

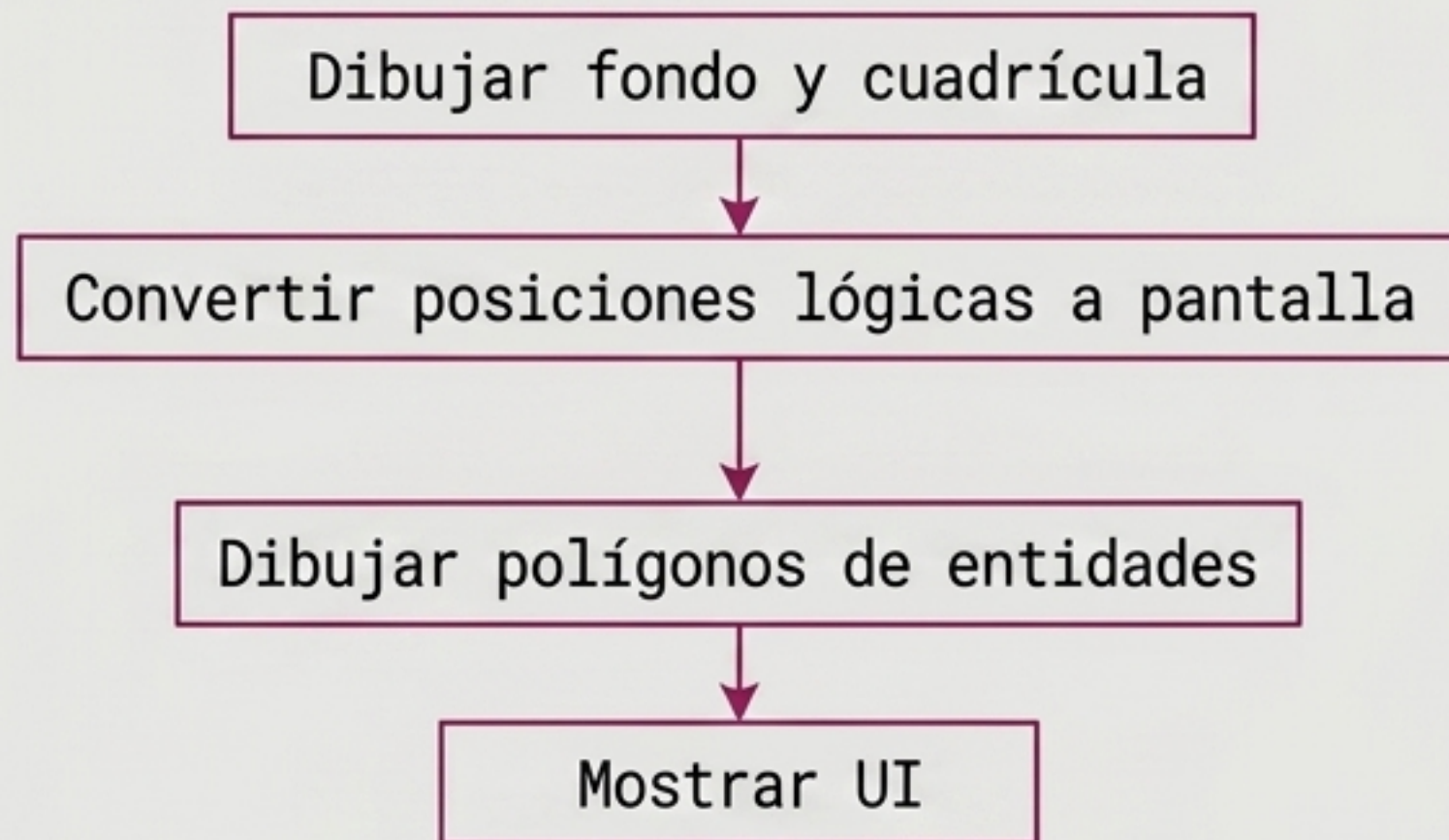
- Sin imágenes, sprites, fuentes ni sonidos.
- Todo se genera dinámicamente mediante las funciones de dibujo de Node2D.
- El objetivo: Crear una representación deliberadamente sencilla para entender la arquitectura subyacente sin la complejidad de la gestión de assets.



El Cerebro Lógico



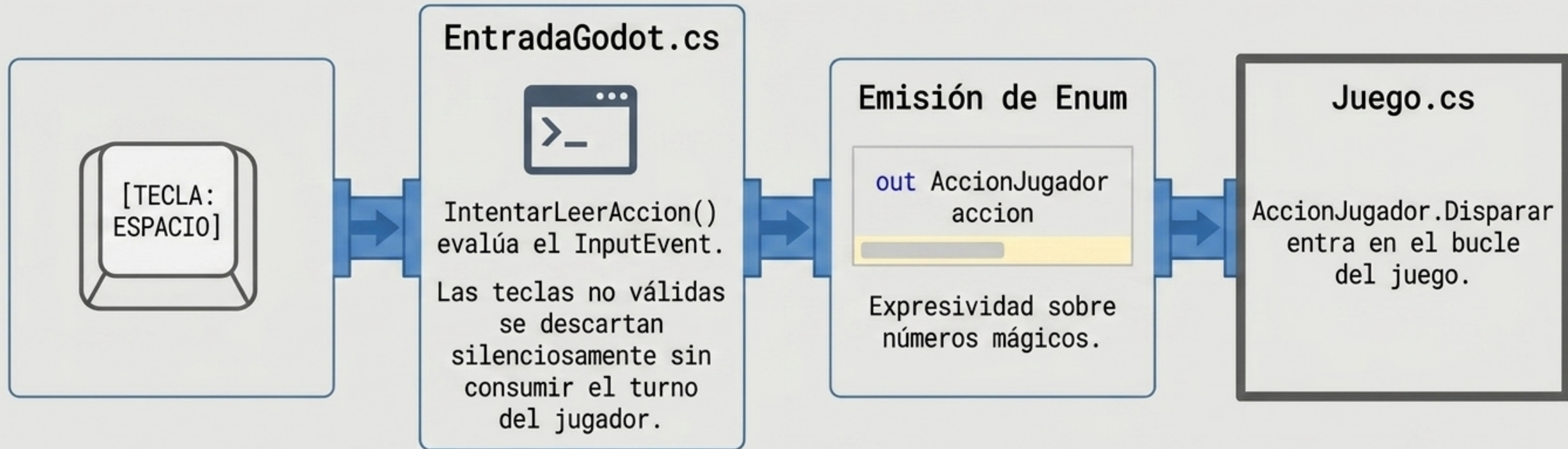
El Motor de Renderizado



```
public partial class Juego : Node2D
```

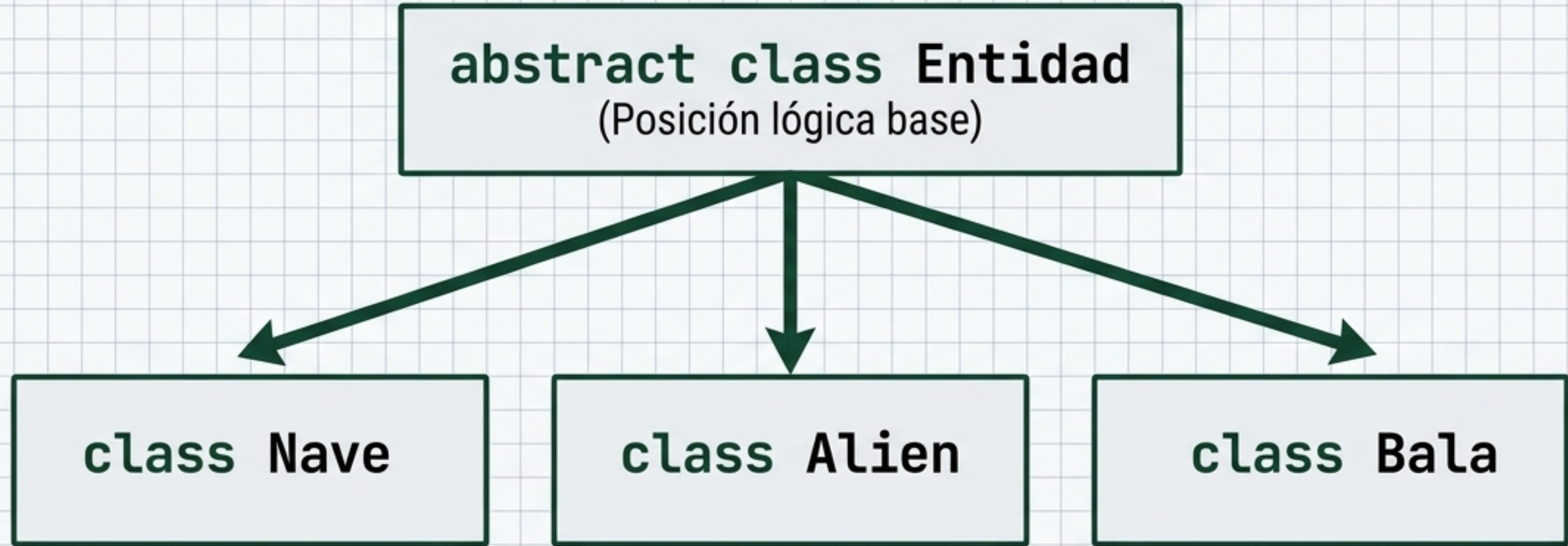
El único punto de contacto real con el sistema de nodos de Godot.

El Flujo de Entrada de Acciones



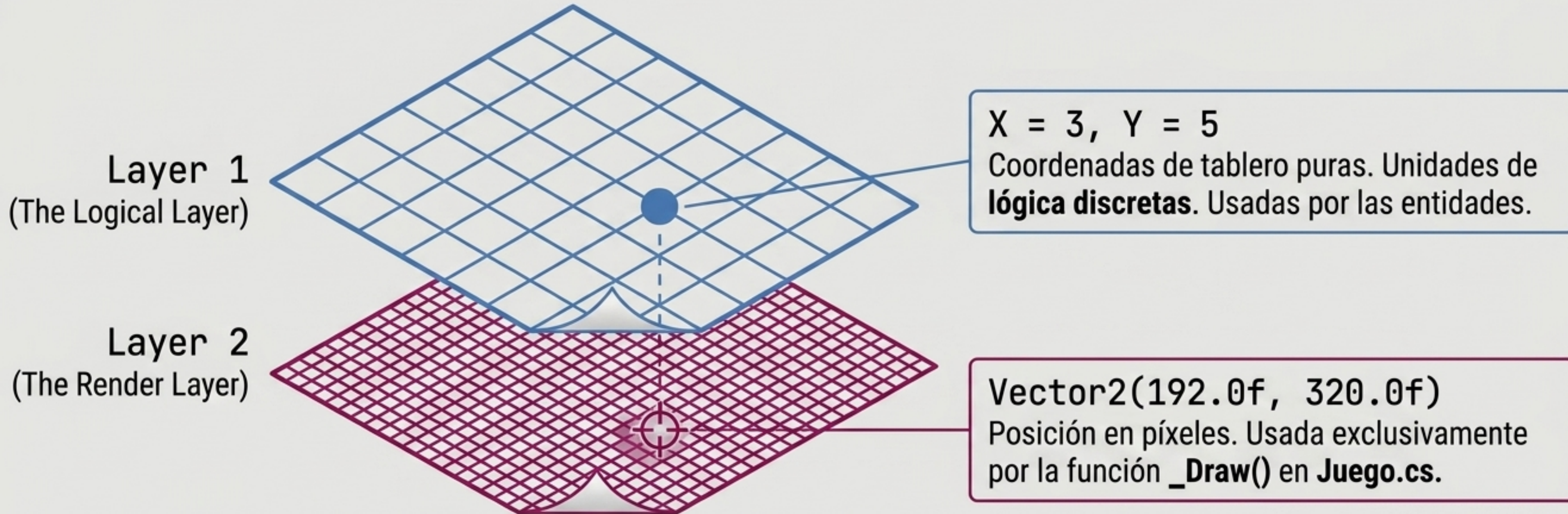
Cambiar una tecla física jamás obliga a modificar la lógica central en Juego.cs.

FRONTERA ESTRICTA: SIN NODOS DE GODOT



Estas clases no heredan de Node2D.
Son clases de C# puro (POCOs) que únicamente almacenan el estado lógico. La lógica del juego no sabe qué es un píxel.

Separación de Coordenadas: Lógica vs. Renderizado



Al aislar las coordenadas lógicas, el diseño visual del juego puede rediseñarse por completo sin alterar ni una sola línea del algoritmo de colisiones o movimiento.

Arquitectura de Entidad: Restricciones en el Código

Evita la instanciación directa. Solo existen naves, alienígenas o balas, nunca entidades genéricas.

```
abstract class Entidad {  
  
    protected Entidad(int x, int y) {  
        X = x; Y = y;  
    }  
}
```

El constructor solo es accesible mediante `base(x, y)` desde las clases derivadas. Impide que agentes externos modifiquen el contrato de creación.

Obliga a definir una posición lógica válida desde el instante de creación.



Aislamiento de Persistencia: GestorRecord.cs

Módulo de Diseño

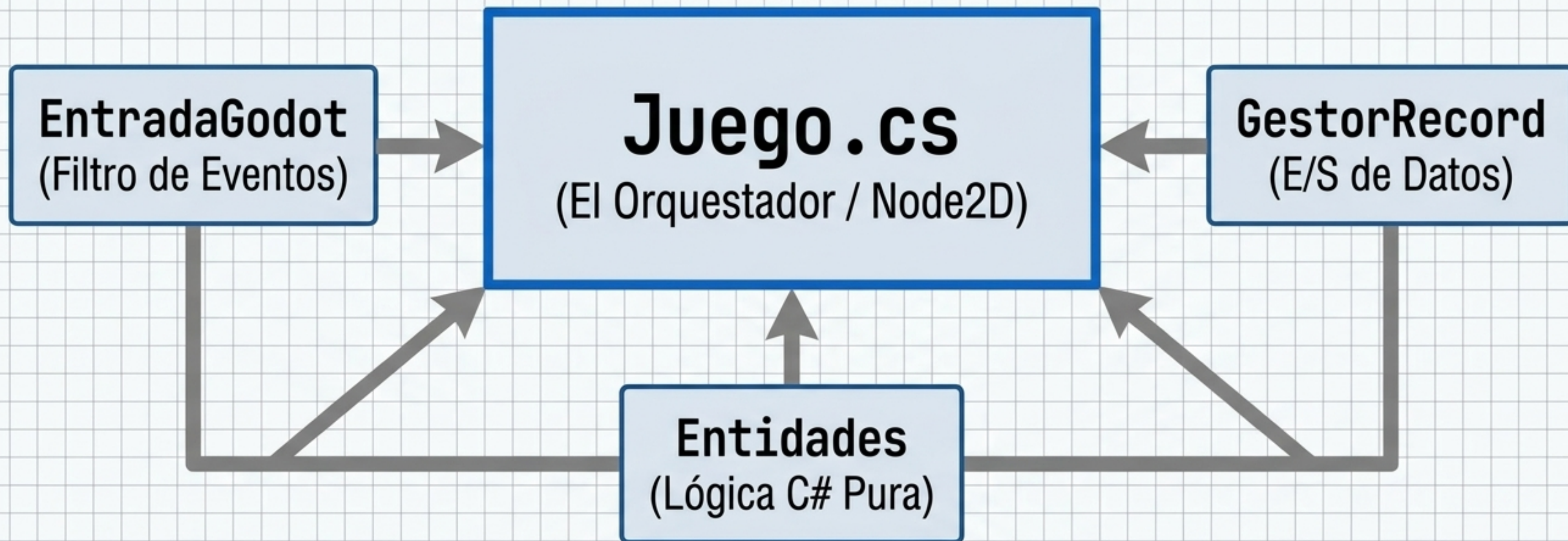
```
static class GestorRecord
```

No se requiere instanciar objetos.
Es un servicio global y puro que solo lee y escribe enteros.
Separar la E/S evita contaminar la lógica de Juego.

Tabla de Comparación de Rutas de Archivo

	res://	Carpeta de recursos del proyecto. De solo lectura en el juego exportado.
	user://record.txt	Carpeta de datos del usuario. Segura, con permisos de escritura garantizados en cualquier plataforma.

Arquitectura Maestra: Inversión de Dependencias



La maestría de esta arquitectura reside en la dirección de las dependencias. Las entidades, la entrada y los registros ignoran por completo la existencia del motor gráfico. **Juego** es el único traductor entre la pureza del código C# y el ecosistema visual de Godot.